

第 01 讲：Long-horizon Coding Agent：问题定义与系统总图

Coding Agent Notes (June 2026)

1 核心图景

Long-horizon coding agent 不是“会写代码的聊天模型”，也不只是一个接了工具调用的代码补全器。它是一个在真实或近似真实的软件环境中持续行动的系统：它要理解目标，选择上下文，修改文件，运行命令，观察失败，把失败转化为下一步决策，并在合适的时候提交一个可验证的结果。



这类系统可以被拆成六个相互耦合的层次：

Long-horizon coding agent = model capability × agent harness × repository context
× tools/runtime × verification feedback × safety/evaluation discipline.

这个公式的重点不是数学乘法，而是系统乘法：任意一层失效，整体能力都会被拉低。强模型如果拿不到关键文件，会写出看似合理但位置错误的 patch；上下文选择正确但编辑工具不稳定，修复也可能没有真正落到文件系统；patch 能跑过局部测试，但如果权限、安全和评估边界不清，仍然不能进入真实生产流程。

把 coding agent 当成分层系统之后，很多争论会变得清楚：一个 benchmark 分数不是纯模型能力，一个工具调用记录不是完整软件工程过程，一个通过测试的 patch 也不必然等价于可维护的工程成果。长期编码能力来自模型、环境和反馈闭环共同作用。

1.1 三个必要条件

长期性不是“上下文窗口更长”，也不是“对话轮数更多”。一个任务要真正成为 long-horizon coding task，至少需要三个条件。

第一，状态会变化。模型不是只在一段 prompt 上生成答案，而是在一个会被修改的工作区里行动。文件内容会变，测试结果会变，假设会被推翻，已经看过的文件和已经尝试过的 patch 也会成为后续决策的一部分。

第二，动作会产生外部后果。一次编辑不是“建议修改”，而是修改真实文件；一次 shell 命令不是“解释命令”，而是产生 stdout、stderr、退出码和副作用；一次浏览器操作不是“描述 UI”，而是改变页面状态。动作必须进入环境，环境必须返回可观察结果。

第三，反馈会约束下一步。测试失败、类型错误、截图差异、CI 日志和 review comment 都不是末尾评分，而是下一轮行动的输入。如果系统不能把反馈压缩、保留并用于更新计划，它只是多轮生成，不是长期工程行动。

Insight. 长期性不等于“对话轮数很多”。真正的长期性来自状态会变化、错误会累积、动作需要执行、反馈会改变下一步决策。没有环境状态和可执行反馈，长上下文也只是更长的 prompt。

2 为什么普通代码生成不够

传统代码模型通常面对的是局部问题：补全一个函数、根据说明写一个算法、修一个独立文件里的 bug。这些任务当然需要代码能力，但它们弱化了真实软件工程里最难的部分：定位问题、理解仓库约束、处理环境反馈、在不破坏其他行为的前提下修改系统。



任务层级	主要状态	典型验证方式
Snippet	局部输入输出	单元测试或样例
Function	函数签名、边界条件	单元测试
File edit	文件内依赖、局部 API	文件级测试
Repo issue	跨文件调用、真实 bug/feature	仓库测试
PR workflow	分支、diff、CI、review	CI + review
Product task	用户目标、权限、UI、部署影响	E2E + 人类验收

任务越往右，越不可能只靠一次文本生成完成。Repo issue 往往要求系统先确定 bug 位置，再读相关调用链，接着修改一处或多处代码，最后运行测试确认为变化。中间任何一步都可能失败，而且失败会污染后续状态。

SWE-bench 把这个边界推进了一步。它从真实 GitHub issue 和 pull request 中构造 2,294 个软件工程问题，覆盖 12 个 Python 仓库。每个任务给出 issue 描述和代码库状态，系统必须提交 patch，并通过仓库测试来评估。这个设计把“会不会写代码”推进到“能不能在仓库状态中修复问题”。

2.1 任务长度不是 token 长度

长上下文可以让模型看到更多文件，但不能自动解决以下问题：哪些文件应该进入上下文，哪些日志应该保留，哪些输出是噪声，哪些测试失败说明方向错了，哪些修改应该回滚。长期任务的核心难点是状态管理，而不是单纯的上下文长度。

例如，一个模型可能在 200K token 窗口里看到了整个仓库，却仍然不知道哪个测试失败最关键；另一个系统可能只给模型 8K token，但通过 repo map、搜索工具和测试反馈不断缩小工作集，反而更稳定。窗口大小只是资源，工作状态才是机制。

2.2 答案不是 patch 字符串

在传统代码生成里，答案常常是一段字符串；在 repository repair 里，答案是工作区状态的变化。一个 patch 要能被应用，要能通过测试，要和周围代码风格兼容，要避免引入新副作用。真正的输出不是“模型说了什么”，而是“仓库变成了什么”。

这也是为什么可执行验证如此重要。没有测试或运行反馈时，系统只能判断 patch 是否看起来合理；有了测试，系统至少能判断某些行为是否被修复。测试仍然不是完美 oracle，但它把软件工程任务从语言相似度拉回到行为约束。

Insight. 长期编码 agent 的关键问题不是“要不要自治”，而是“哪些决策交给模型，哪些决策交给系统”。Agentless、SWE-agent 和 OpenHands 的差异，本质上是决策边界的差异。

3 Benchmark 是压缩世界

Benchmark 的价值在于压缩真实世界。真实软件工程包含需求澄清、设计讨论、代码审查、权限控制、CI、部署、监控和长期维护；benchmark 会把其中一部分固定下来，让不同系统可以在同一组任务上比较。



这种压缩既是优点，也是风险。优点是可复现：同一个 repo snapshot、同一个 issue、同一套测试，可以让研究者比较不同系统。风险是被压缩掉的部分会重新进入真实部署：用户意图可能不完整，环境可能不可复现，测试可能不覆盖关键行为，权限和安全边界可能比 benchmark 复杂得多。

3.1 SWE-bench 压缩了什么

SWE-bench 把真实 issue 修复压缩成一个可评测实例：给定仓库快照和 issue 描述，系统生成 patch，评测端运行测试判断是否解决。它保留了 repository context 和 executable tests，因此比 HumanEval 这类单函数任务更接近软件工程。

但它也刻意排除了很多产品环境因素：开发者和用户之间的澄清对话，代码审查中的风格争议，CI flakes，权限审批，部署后监控，以及长期维护。理解这点很重要：SWE-bench 是研究长期编码能力的关键基准，但不是完整软件工程世界。

3.2 压缩导致的读数偏差

如果一个 agent 在 benchmark 上得分高，可能有三种解释：模型更会理解代码，harness 更会给模型提供有效行动路径，或者 benchmark 的压缩形式正好适配这个系统。第三种情况尤其危险，因为它会把 benchmark optimization 误读成 general engineering ability。

例如，强 patch validation 可以显著提高最终提交质量，但这不等于模型第一次生成 patch 的质量更高；强检索器可以把关键文件放到模型面前，但这不等于模型自己具备完整 repo navigation 能力；更大的 rollout budget 可以扩大搜索空间，但这不等于单次决策更可靠。

Insight. Benchmark 不是现实的缩略图，而是现实的有损压缩。读 benchmark 结果时，必须同时问：保留了哪些工程约束，删除了哪些工程约束，系统是否正在利用这个压缩方式。

4 系统的六个层次

一个长期编码 agent 可以从六层理解。每一层都回答一个不同问题：模型能想什么，harness 允许它怎样行动，context 让它看见什么，runtime 让动作怎样发生，verification 怎样判断反馈，safety/evaluation 怎样让系统可控、可比较、可部署。

4.1 Model

模型层提供基础能力：理解自然语言 issue、阅读代码、推理跨文件依赖、生成 patch、解释测试失败、规划下一步。没有足够强的 foundation model，harness 再好也只是把弱能力包装得更复杂。

但模型能力只能决定“可能做什么”，不能单独决定“最终做成什么”。模型可能知道应该修改某个函数，却不会稳定地在仓库里找到它；模型可能理解错误原因，却被过长日志淹没；模型可能生成了正确思路，却因为编辑工具格式不稳定而没有应用成功。

模型层的典型失败是语义和推理失败：误解 issue、忽略约束、改错 API、写出通过局部测试但破坏全局行为的 patch。系统设计不能把这些失败全推给 harness；弱模型确实会限制上限。但同样不能把所有成功都归因给模型，因为成功需要经过后续五层转化。

4.2 Harness

Harness 是模型和计算机之间的工作制度：prompt 怎么组织，动作怎么表示，工具怎么调用，观察怎么返回，什么时候停止，失败怎么处理。SWE-agent 把其中一部分称为 agent-computer interface (ACI)，强调 LM agent 是一种新的 end user，需要为它设计专门的界面。

Harness 不是外壳。它决定模型的能力如何被转化为行动。一个好的 harness 会把搜索、查看、编辑、测试这些动作设计成模型容易使用、系统容易审计的接口；一个差的 harness 会让模型在自由 shell、长日志、脆弱 diff 和模糊停止规则之间消耗能力。

Harness 的核心 tradeoff 是自由度和可控性。自由度太低，模型无法探索复杂修复；自由度太高，模型容易乱试、误读反馈、执行危险动作或无限循环。Agentless 展示了另一种选择：把 localization、repair、validation 交给结构化 pipeline 控制，让模型只在每个子问题里发挥能力。

4.3 Repository Context

Repo context 不等于把整个仓库塞进上下文窗口。真正的问题是选择、压缩和更新：哪些文件相关，哪些符号需要展开，哪些测试失败值得保留，哪些历史操作会影响下一步。

在 repo issue 中，问题常常跨越 issue 描述、测试文件、实现文件、配置文件和历史约定。Context selector 如果漏掉关键测试，模型会纠错行为；如果塞入太多无关文件，模型会被噪声干扰；如果不能保留“已经尝试过什么”，系统会重复失败路径。

因此，context 是动态工作集，而不是静态 prompt。每次工具调用、测试失败和编辑结果都可能改变下一轮需要看的内容。长期任务中的 context engineering，本质上是在有限注意力里维护一个不断更新的工程现场。

4.4 Tools / Runtime

Tools 决定动作空间，runtime 决定动作是否真实发生。CodeAct 把 executable Python code 作为 action space，动机是让 agent 能组合工具、调用库，并从执行错误中调整下一步。OpenHands 则提供 docker sandbox、bash、browser、IPython server 等运行层，让 agent 的动作可以被观察、记录和约束。

工具设计的关键不是“工具越多越好”。工具越强，动作空间越大，模型越可能解决真实任务；但工具越强，副作用、权限、审计和错误恢复也越难。自由 shell 可以做几乎所有事，也可以删错文件、联网泄漏、跑很久或产生难以解释的输出。结构化编辑工具更可控，但可能限制复杂修改。

Runtime 还决定复现性。一个 patch 在本地能跑，不代表在 clean container 中能跑；一个测试失败可能来自代码逻辑，也可能来自依赖缺失、环境变量、系统时间或 flaky external service。长期 agent 必须把 runtime 当作状态的一部分，而不是把它视为透明背景。

4.5 Verification

长期编码任务必须有反馈闭环。SWE-bench 通过 tests 评估 patch；Agentless 在 patch validation 阶段筛选候选修复；Qwen3-Coder-Next 把可执行验证用于大规模训练任务。没有 verification，agent 很容易只是在生成看起来合理的代码。

Verification 的作用有两层。第一层是评估：最终 patch 是否满足 oracle。第二层是控制：中间失败能否给下一轮提供方向。很多 agent 失败不是因为完全不会写代码，而是因为无法把测试失败解释成

下一步行动，或者只根据局部测试过拟合。

测试不是完美 oracle。测试可能不完整，可能过拟合，可能被 reward hacking 利用。但如果完全没有可执行验证，软件工程任务就退化成本评分。好的 verification 设计要承认测试的局限，同时让测试成为可观察、可复现、可用于调试的反馈源。

4.6 Safety / Evaluation

当 agent 可以执行命令、修改文件、访问网络、运行浏览器时，权限和安全就不是产品附属功能，而是系统层需求。长期任务的风险还包括 benchmark leakage、reward hacking、无效工具调用、破坏性操作、隐私泄漏和成本失控。

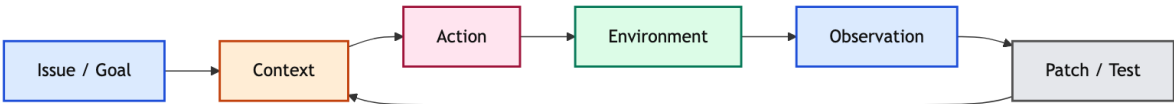
Evaluation 也不应只记录 pass rate。一个系统可能通过更多 rollout、更大预算、更强 validation 取得高分，但成本过高、轨迹不可审计或权限过宽。另一个系统分数稍低，但失败类型清楚、成本稳定、权限边界严格，更适合真实部署。

安全和评估层回答的是“这个系统能否被信任地运行”。这包括能否复现结果，能否解释失败，能否限制副作用，能否在人类需要介入时暂停，能否防止 agent 利用环境漏洞绕过真实任务。

层次	关键问题	缺失后的表现
Model	能否理解、推理、生成代码	语义正确性不足
Harness	能否稳定行动与停止	乱试、格式错、无限循环
Context	能否看到正确工作集	漏文件、噪声过多
Runtime	动作是否真实且可复现	环境失败、编辑未生效
Verification	反馈是否可靠可用	看似合理但不可验证
Safety/Eval	是否可控、可比较、可部署	高分但不可审计

5 Agent loop：为什么必须闭环

把上述层次放到一次任务执行里，可以得到一个最小闭环。



这个 loop 的核心不是“模型想一步，工具做一步”这么简单。每一轮都会改变工作状态：context 被更新，文件被编辑，测试产生新失败，模型的假设被强化或推翻。长期任务的难点来自这种状态传播。

Loop 节点	作用	典型失败
Issue / Goal	给出目标和约束	需求歧义、目标过宽
Context	找到相关代码和历史状态	漏文件、上下文污染
Action	编辑、搜索、运行命令	工具格式错、动作过大
Environment	文件系统、shell、browser、tests	环境不稳定、依赖缺失
Observation	stdout、diff、测试失败、截图	反馈太长或不可解释
Patch / Test	产生可验证状态变化	过拟合测试、修错位置

短任务允许模型一次性输出答案；长期任务要求系统持续把观察压缩成下一轮可用的 context。这

个压缩过程很难：stdout 可能很长，测试失败可能有多个，diff 可能影响多处文件，模型可能需要保留“这个方向已经试过且失败”的负信息。

5.1 一次 repo repair 的典型闭环

设想一个 issue 说某个 parser 在遇到空输入时行为错误。一个长期编码 agent 的合理轨迹通常不是直接写 patch，而是经历以下阶段：先搜索错误信息和相关测试；再打开可能相关的 parser 文件；接着运行局部测试复现失败；然后修改边界条件；再运行测试；如果测试暴露另一个调用路径，再回到 context 选择阶段。

这个过程里的每一步都不大，但合起来就是长期任务。真正的能力体现在：系统能否记住已经看过哪些文件，能否把测试失败定位到代码路径，能否区分新失败和旧失败，能否在 patch 变坏时回滚，能否在证据足够时停止。

5.2 为什么错误会累积

长期任务的坏处是错误会传播。第一次 context 选错，计划就会偏；计划偏了，编辑就会修错位置；修错位置后，测试反馈可能变得更混乱；系统如果把混乱反馈写进记忆，后续行动会越来越自信地走错方向。



这也是为什么 trajectory integrity 很重要。一个完整轨迹不仅包括最终 patch，还包括目标、上下文、工具调用、观察结果、失败、回退和停止。训练和评估如果只看最终答案，会丢掉大量关于“系统怎样走到答案”的信息。

Insight. 一个 coding agent 的真实能力，常常不是由最好的一步决定，而是由最差的一步限制。一次错误定位、一次 malformed tool call、一次误读测试失败，都可能让后续 20 轮看似努力但方向错误。

6 自治不是目标，决策边界才是

长期任务很容易让人联想到“让 agent 自己做所有决定”。但系统设计上，更重要的问题不是自治程度，而是决策边界：哪些决策由模型做，哪些决策由 harness、pipeline、检索器、验证器或人类做。

全自治 loop 的优势是灵活。模型可以根据观察动态决定下一步，适合任务开放、线索复杂、需要探索的场景。缺点是错误空间大：模型可能乱试工具、误读反馈、跳过验证、过早停止或无限循环。

结构化 pipeline 的优势是可调试。Agentless 把任务拆成 localization、repair、patch validation，降低模型在复杂工具空间中自由游走的风险。缺点是流程可能限制探索，也可能把错误固化在早期阶段：如果 localization 错了，后续 repair 再强也可能修错地方。

人类在环 的优势是安全和意图对齐。真实产品任务里，人类可以确认目标、审批高风险动作、选择 patch、解释业务约束。缺点是吞吐下降，系统必须设计清晰的交互点，否则人类只会被大量低质量中断打扰。

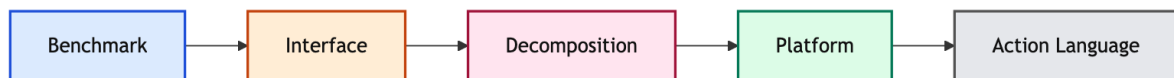
决策方式	适合什么	主要风险
全自治 loop	开放探索、多步调试	错误传播、成本失控
结构化 pipeline	可分解 repo repair	早期错误被固化
强验证筛选	多候选 patch 选择	把验证误读成生成能力
人类在环	高风险产品任务	交互成本和打扰

因此，一个好的长期编码系统不一定最自治，而是把不同类型的决策放在最合适的位置。模型擅长语义理解、代码推理和候选生成；系统擅长记录状态、执行验证、限制权限和复现流程；人类擅长判断业务意图、风险和最终可接受性。

Insight. Autonomy 是一种资源，不是一种目标。把所有决策都交给模型，会扩大灵活性，也会扩大错误传播面。系统设计的关键是把决策放在最能承担后果的一层。

7 五种代表性设计路线

不同研究和系统把长期编码问题切在不同位置。把它们放在一张图里，可以看到它们不是互相替代，而是在强调系统的不同层。



7.1 SWE-bench: 任务和 oracle 路线

SWE-bench 的核心贡献是把真实 repo issue 变成可执行评测实例。它不是一个 agent 系统，而是给 agent 系统提供任务边界和 oracle：你是否能在代码库里产生 patch，并通过测试验证。

这条路线的启发是：如果没有可执行任务，长期编码能力就很难被稳定比较。但它的限制也很清楚：测试 oracle 有覆盖边界，benchmark split 有分布边界，环境构造有复现边界。评测越可控，就越可能漏掉真实产品中的开放因素。

7.2 SWE-agent: 界面路线

SWE-agent 强调 agent-computer interface。它说明，同一个模型面对不同动作接口会表现不同。raw shell 对人类工程师很自然，但对 LM agent 未必是好界面；查看、搜索、编辑和测试反馈需要被组织成模型更容易使用的命令和观察格式。

这条路线的启发是：模型能力必须通过界面显现。一个强模型如果工具输出太长、编辑格式太脆、命令反馈太乱，能力会浪费在接口摩擦上。界面不是美化层，而是能力转换层。

7.3 Agentless: 分解路线

Agentless 反过来质疑复杂自治 agent 是否必要。它用 localization、repair、patch validation 这样的结构化流程减少自由探索，让模型在更小的子问题里工作。它展示了一个重要事实：长期任务不一定需要一个完全自由行动的 agent；外部 pipeline 也可以承担规划和控制。

这条路线的启发是：系统可以用约束换稳定。把任务分解得好，模型不必在巨大动作空间里搜索；把 validation 做得清楚，候选 patch 可以被筛掉。缺点是结构化流程可能牺牲灵活性，也可能把复杂任务硬切成不自然的子任务。

7.4 OpenHands: 平台路线

OpenHands 把 coding agent 放进完整平台: event stream、docker sandbox、bash、browser、IPython server、多 agent delegation、evaluation framework。它的意义不是“工具更多”，而是说明长期 agent 需要一个可运行、可记录、可约束的环境。

这条路线的启发是：一旦 agent 要离开 benchmark script，平台层就会变成核心。权限、沙箱、浏览器、事件流、评估接口和人工介入都不是装饰，它们决定 agent 是否能在真实任务里持续、安全、可复现地工作。

7.5 CodeAct: 动作语言路线

CodeAct 关注 action representation。用 executable Python code 作为动作空间，可以让 agent 组合库、调用函数、写小程序处理观察结果，并根据执行错误继续调整。这比固定 JSON tool call 更有表达力。

但动作语言越强，控制难度越高。可执行代码能表达复杂计划，也能产生副作用；能组合工具，也更难审计；能利用库，也可能触碰权限边界。因此，CodeAct 的启发不是“代码动作永远更好”，而是动作语言本身就是系统设计变量。

8 最小可用规格

如果把定义落到工程规格上，一个系统至少要满足下面六个条件，才值得称为 long-horizon coding agent。否则它可能只是代码补全器、批处理脚本、静态修复器，或者带工具调用的聊天模型。

规格	最低要求	缺失后的退化
目标状态	有明确 issue、task 或用户目标	变成开放聊天
工作状态	能维护文件、假设、历史观察	每轮像重新开始
行动接口	能搜索、编辑、运行或调用工具	只能建议代码
环境反馈	能读取测试、命令、diff 或截图	只能主观判断
停止规则	有完成、失败、回滚或升级条件	容易无限探索
安全边界	有权限、sandbox 或审计机制	难以真实部署

这张表也解释了为什么很多 demo 看起来像 agent，但不能稳定处理长期任务。它们可能调用工具，却没有可靠停止规则；可能有长上下文，却没有状态压缩；可能跑测试，却没有权限边界。长期任务一旦进入真实代码库，这些缺口会迅速放大。

8.1 四个容易混淆的边界

Long context 不等于 long horizon。 长上下文只解决“能看多少”。长期任务还要解决“看完以后如何行动，行动以后如何观察，观察以后如何更新状态”。

Tool use 不等于 agent。 一次函数调用或一次 shell 命令只是动作。agent 需要把动作纳入目标、上下文、反馈和停止规则构成的闭环。

Autonomy 不等于 intelligence。 Agentless 说明，外部 pipeline 可以减少模型自由探索的空间，反而提高任务完成率和可调试性。

Passing tests 不等于 engineering success。 测试通过说明 patch 满足当前 oracle，但还不能证明代码可维护、没有副作用、适合合并或符合用户真实意图。

Insight. 一个实用的判断标准是：如果把模型输出接到真实代码库、真实工具和真实验证上，这个系统是否还能被观察、恢复、约束和复现。能，才接近 long-horizon coding agent；不能，就仍然主要是文本生成。

9 结果怎么读

不要把一个 pass rate 当作单点事实，而要把它放进四个坐标里。

坐标	要问的问题	例子
任务分布	测的是什么任务？	SWE-bench full / Lite / Verified
系统边界	模型之外还有哪些 harness？	ACI、检索、patch validation
资源成本	每个任务花多少 token、时间、rollout？	多候选 patch、长轨迹、并行测试
失败类型	没通过时失败在哪里？	定位错、编辑错、测试误读、工具失败

SWE-agent 的贡献不是单纯“某个模型分数更高”，而是说明 ACI 会改变 agent 能力的可见上限。Agentless 的贡献也不是简单“少自治更好”，而是说明复杂自主 loop 不是唯一可行路线。OpenHands 的贡献不只是提供一个平台，而是把 sandbox、event stream、browser、shell、evaluation 都纳入可复现系统。

因此，看到任何结果，都应该同时追问：模型是谁，harness 是什么，context 怎么来，工具能做什么，验证怎样执行，失败是否可复现。缺少这些信息的分数很难比较。

9.1 四种常见误读

把模型提升误读成系统提升。 如果模型和 harness 同时换了，分数只能说明系统变强，不能单独说明模型变强。

把资源预算误读成能力提升。 更多候选 patch、更多 rollout、更多验证轮次都可能提高结果，但它们也提高成本。比较结果时必须看 token、时间和候选数。

把验证筛选误读成一次生成质量。 如果系统生成很多候选再筛选，最终 patch 质量高不代表第一次生成质量高。验证是系统能力的一部分。

把 benchmark 成功误读成产品成功。 Benchmark 环境通常固定且受控；产品环境包含权限、用户上下文、未提交代码、UI 状态、审查和部署风险。两者相关，但不等价。

10 局限与开放问题

Long horizon 不是越长越好。 更多轮交互可能带来更多修复机会，也可能带来更多错误累积。一个强 pipeline 有时会比一个自由行动的 agent 更可靠。

Tests 是必要但不充分的 oracle。 可执行测试让任务可验证，但测试覆盖率、隐藏用例、环境稳定性和 benchmark 构造质量都会影响结果。通过测试不等于代码长期可维护。

Harness 可能过拟合 benchmark。 如果 agent 的工具、prompt、搜索策略只为某个 benchmark 调优，leaderboard 分数可能高于真实产品可用性。这不意味着 benchmark 无用，而是要求报告清楚系统边界、预算、验证方式和失败类型。

部署约束会反过来改变研究问题。 成本、延迟、权限、审计、隐私、用户交互和 UI 验证，都会改变 agent 应该如何设计。一个研究 harness 可以只追求 pass rate；产品级 agent 还必须能被人监督和恢复。

11 最终 takeaway

长期编码 agent 是一个分层系统，不是一个单独模型。

它的长期性来自持续变化的 repository state、可执行环境反馈，以及能把失败带回下一轮决策的闭环。

判断一个 coding agent，不能只问“模型多强”。还要问：它如何选择上下文，如何行动，如何接收反馈，如何验证，如何限制副作用，如何在失败后恢复。

参考资料

- [1] [Agent4SE Survey](#). survey taxonomy
- [2] [SWE-bench](#). repo-level benchmark
- [3] [SWE-agent](#). agent-computer interface
- [4] [Agentless](#). localization-repair-validation
- [5] [OpenHands](#). open software-agent platform
- [6] [CodeAct](#). executable code actions